

CS3237 Lab 4 - Neural Networks

[Student Number|Name | |:------|:------| | | Wira Azmoon Ahmad | | | R Ramana |

Please work together as a team of 2 to complete this lab. You will need to submit ONE copy of this notebook per team, but please fill in the names of both team members above. This lab is worth 25 marks:

DO NOT SUBMIT MORE THAN ONE COPY OF THIS LAB!

1. Introduction

The objectives of this lab are:

1. To familiarize you with how to create dense neural networks using Keras.
2. To familiarize you with how to encode input and output vectors for neural networks.
3. To give you some insight into how hyperparameters like learning rate and momentum affect training.

To save time we will train each experiment only for 50 epochs. This will lead to less than optimal results but is enough for you to make observations.

HINT: YOU CAN HIT SHIFT-ENTER TO RUN EACH CELL. NOTE THAT IF A CELL IS DEPENDENT ON A PREVIOUS CELL, YOU WILL NEED TO RUN THE PREVIOUS CELL(S) FIRST

2. The Irises Dataset

We will now work again on the Irises Dataset, which we used in Lab 3, for classifying iris flowers into one of three possible types. As before we will consider four factors:

1. Sepal length in cm
2. Sepal width in cm
3. Petal length in cm
4. Petal width in cm

In this dataset there are 150 sample points. The code below loads the dataset and prints the first 10 rows so we have an idea of what it looks like.

```
In [ ]: from sklearn.datasets import load_iris
        from sklearn.preprocessing import MinMaxScaler
        from sklearn.model_selection import train_test_split
```

```
iris = load_iris()

print("First 10 rows of data:")
print(iris.data[:10])
```

```
First 10 rows of data:
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]
 [5.4 3.9 1.7 0.4]
 [4.6 3.4 1.4 0.3]
 [5.  3.4 1.5 0.2]
 [4.4 2.9 1.4 0.2]
 [4.9 3.1 1.5 0.1]]
```

2.2 Scaling the Data

We make use of the `MinMaxScaler` to scale the inputs to between 0 and 1. The code below does this and prints the first 10 rows again, to show us the difference.

In the next section we will investigate what happens if we use unscaled data.

```
In [ ]: scaler = MinMaxScaler()
scaler.fit(iris.data)
X = scaler.transform(iris.data)

print("First 10 rows of SCALED data.")
print(X[:10])
```

```
First 10 rows of SCALED data.
[[0.22222222 0.625      0.06779661 0.04166667]
 [0.16666667 0.41666667 0.06779661 0.04166667]
 [0.11111111 0.5        0.05084746 0.04166667]
 [0.08333333 0.45833333 0.08474576 0.04166667]
 [0.19444444 0.66666667 0.06779661 0.04166667]
 [0.30555556 0.79166667 0.11864407 0.125      ]
 [0.08333333 0.58333333 0.06779661 0.08333333]
 [0.19444444 0.58333333 0.08474576 0.04166667]
 [0.02777778 0.375     0.06779661 0.04166667]
 [0.16666667 0.45833333 0.08474576 0.      ]]
```

2.3 Encoding the Targets

In Lab 3 we saw that the target values (type of iris flower) is a vector from 0 to 2. We can see the 150 labels below:

```
In [ ]: print(iris.target)
```

```
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0  
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0  
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1  
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2  
2 2]
```

We can use this to train the neural network, but we will use "one-hot" encoding, where we have a vector of n integers consisting of 0's and 1's. The table below shows how one-hot encoding works:

Value	One-Hot Encoding
0	[1 0 0]
1	[0 1 0]
2	[0 0 1]

Keras provides the `to_categorical` function to create one-hot vectors:

```
In [ ]: from keras.utils import to_categorical

Y = to_categorical(y = iris.target, num_classes = 3)
print(Y)
```

Now let's split the data into training and testing data:

[illegible]

2.4 Building our Neural Network

Let's now begin building a simple neural network with a single hidden layer, using the Stochastic Gradient Descent (SGD) optimizer, ReLu transfer functions for the hidden layer and softmax for the output layer.

The code to do this is shown below:

```
In [ ]: from keras.models import Sequential
        from keras.layers import Dense
        from keras.optimizers import SGD
```

```
# Create the neural network
nn = Sequential()
nn.add(Dense(100, input_shape = (4, ), activation = 'relu'))
nn.add(Dense(3, activation = 'softmax'))

# Create our optimizer
sgd = SGD(lr = 0.1, momentum=1)

# 'Compile' the network to associate it with a loss function,
# an optimizer, and what metrics we want to track
nn.compile(loss='categorical_crossentropy', optimizer=sgd,
           metrics = 'accuracy')
```

2.5 Training the Neural Network

As is usually the case, we can call the "fit" method to train the neural network for 50 epochs. We will shuffle the training data between epochs, and provide validation data.

```
In [ ]: nn.fit(X_train, Y_train, shuffle = True, epochs = 50,
              validation_data = (X_test, Y_test))
```

Question 1

Run the code above. Do you see evidence of underfitting? Overfitting? Justify your answers. ***(4 MARKS)***

Answer: There is no significant evidence of underfitting or overfitting. There is no real difference between the loss in the training set and validation set. The validation accuracy also increases alongside the training accuracy. If there was overfitting, the loss on the validation set should be much higher with the accuracy being lower. Similarly if there was underfitting, there should be low accuracy and high loss for the validation set. But we see no such evidence.

(For TA) Marks awarded: ____ / 4

Question 2a

Consult the documentation for the SGD optimizer [here](#). What does the lr parameter do? ***(1 MARK)***

Answer: It is the learning rate of the optimizer, and adjusts how big the steps are when updating the weights.

Question 2b

The documentation states that the momentum parameter "accelerates gradient descent in the relevant direction and dampens oscillations". Using Google or other means, illustrate what this means. ***(2 MARKS)***

Answer: Momentum will resist changes in the direction of the gradient change, preferring to continue moving in a certain direction. This is similar to applying the breaks on a car, whereas without momentum the car can change direction at every time step.

(For TA) Marks awarded: ____ / 3

Question 3a

We will now play with the lr parameter. Adjust the lr parameter to the following values and record the final training and validation accuracies in the respective columns. Also observe the sequence of accuracies over the training period, and place your observation in the "remarks" column, e.g. "Progresses steadily", "some oscillation" etc. ***(3 MARKS)***

Answer: Fill the table below

lr	Training Acc.	Validation Acc.	Remarks
0.01	0.6917	0.5667	Underfitting, slow progress
0.1	0.9333	0.9333	Progresses steadily
1.0	0.9500	0.9667	Some oscillation
10.0	0.3083	0.4333	Underfits, cannot find global optima
100	0.3583	0.3667	Underfits, cannot find global optima
1000	0.3083		

0.2000 | Underfits, cannot find global optima | | 10000 | 0.3333 | 0.4333 | Underfits, cannot find global optima | | 100000 | 0.3000 | 0.4333 | Underfits, cannot find global optima |

Question 3b

Based on your observations above, comment on the effect of small and very large learning rates on the learning. ***(2 MARKS)***

Answer: Small learning rates can make training very slow, and take a much longer time to converge. Large learning rates can mean the model does not learn much from the data, since the steps are too large and easily miss the optimal minimums.

(For TA) Marks awarded: ____ / 5

2.5 Using Momentum

We will now experiment with the momentum term. To do this:

1. Change the learning rate to 0.1.
2. Set the momentum to 0.1. Note: Do not use the Nesterov parameter - Leave it as False.

Run your neural network.

Question 4a

Keeping the learning rate at 0.1, complete the table below using the momentum values shown. Again record any observations in the "Remarks" column. ***(3 MARKS)***

Answer: Fill the table below

momentum	Training Acc.	Validation Acc.	Remarks
0.001	0.9417	0.9667	Progresses steadily
0.01	0.9417	0.9667	Progresses steadily
0.1	0.9583	0.9667	Progresses steadily
1.0	0.9500	1.0000	Quickly converges

Question 4b

Based on your observations above, does the momentum term help in learning? ***(2 MARKS)***

Answer: Momentum can speed up learning and prevent getting stuck in local minimums, making it more likely to be in a global minimum.

(For TA) Marks awarded: ____ / 5

2.6 Using Raw Unscaled Data

We begin by using unscaled X and Y data. The code below will create 120 training samples and 30 testing samples (20% of the total of 150 samples):

```
In [ ]: X_unscaled = iris.data
Y_raw = iris.target
X_utrain, X_uteat, Y_utrain, Y_uteat = train_test_split(X_unscaled, Y,
                                                         test_size = 0.2,
                                                         random_state = 1)
```

Question 5

Create a new neural network called "nn2" below using a single hidden layer of 100 neurons. Train using the data in X_utrain, X_uteat and validate with Y_utrain and Y_uteat. Again use the SGD optimizer with a learning rate of 0.1 and no momentum,

and train for 50 epochs. ***(3 marks)***

```
In [ ]: """
Enter your code for Question 5 below. To TA: 3 marks for code.
"""

# Create the neural network
nn2 = Sequential()
nn2.add(Dense(100, input_shape = (4, ), activation = 'relu'))
nn2.add(Dense(3, activation = 'softmax'))

# Create our optimizer
sgd = SGD(lr = 0.1)

# 'Compile' the network to associate it with a Loss function,
# an optimizer, and what metrics we want to track
nn2.compile(loss='categorical_crossentropy', optimizer=sgd,
            metrics = 'accuracy')
```

```
In [ ]: nn2.fit(X_utrain, Y_utrain, shuffle = True, epochs = 50,
              validation_data = (X_uteat, Y_uteat))
```

(Question 5 continues)

Observe the training and validation error. Does not scaling the input affect the training? Why do you think this is so? What is the advantage of scaling? ***(5 MARKS)***

Answer: Yes, it affects training negatively. This is because some of the features can end up dominating the others simply by being bigger values. This can happen when different parameters are of differing distributions. Scaling helps equalize the effect each feature value can have on the neural network.

(For TA) Marks awarded: ____ / 8

3 Conclusion

In this lab we saw how to create a simple Dense neural network to complete the relatively simple task of learning how to classify irises according to their sepal and petal characteristics.

FOR TA ONLY

| Question | Marks | | :-----: | :-----: | | 1 | /4 | | 2 | /3 | | 3 | /5 | | 4 | /5 | | 5 | /8 | | Total: | /25 |